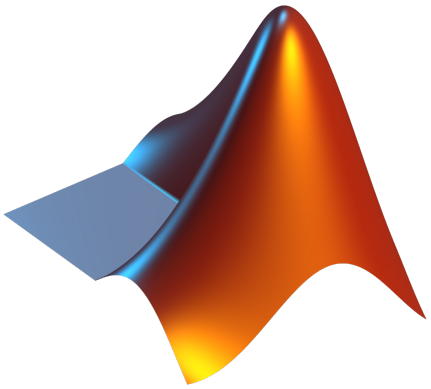
Matrices en MATLAB y Respuesta Transitoria

Fabrizio Benavidez[1]

[1] *Facultad de Ciencias Exactas, Físicas y Naturales, Ingeniería Aeronáutica*

Resumen En el siguiente trabajo se realiza código MATLAB para calcular el lugar de raíces para distintos tipos de sistemas, así como funciones de lazo cerrado. Además, se trabaja por primera vez con funciones: fragmentos de código que se pueden declarar las veces necesarias. Se crean dos tipos de funciones: uno para calcular raíces cuadradas y otras para calcular factor de amortiguamiento, frecuencia y ganancia.

Palabras claves: lugar de raíces, compensadores, funciones



4 de octubre de 2026

Semana 6

[1] fabrizio.benavidez@mi.unc.edu.ar

Guía 11: Compensadores proporcionales

Sea la función G:

$$G(s) = \frac{s^2 + s + 1}{s^3 + 0{,}5s^2 + 3} \tag{1}$$

La razón por la que el sistema es inestable para $H = 1$ es porque tiene polos con parte real positiva, por lo tanto el sistema no es estable. En cambio, cuando $H = 6$, el sistema se hace estable ya que los polos se mueve del semiplano derecho del plano complejo al izquierdo, representando estabilidad.

Ahora, sea el mismo sistema pero el compensador H cambia de valor según el bucle `for c = 3:0.5:9`. La Figura 1 muestra el lugar de raíces (discreto) para el sistema definido por G cuando H cambia de valor:

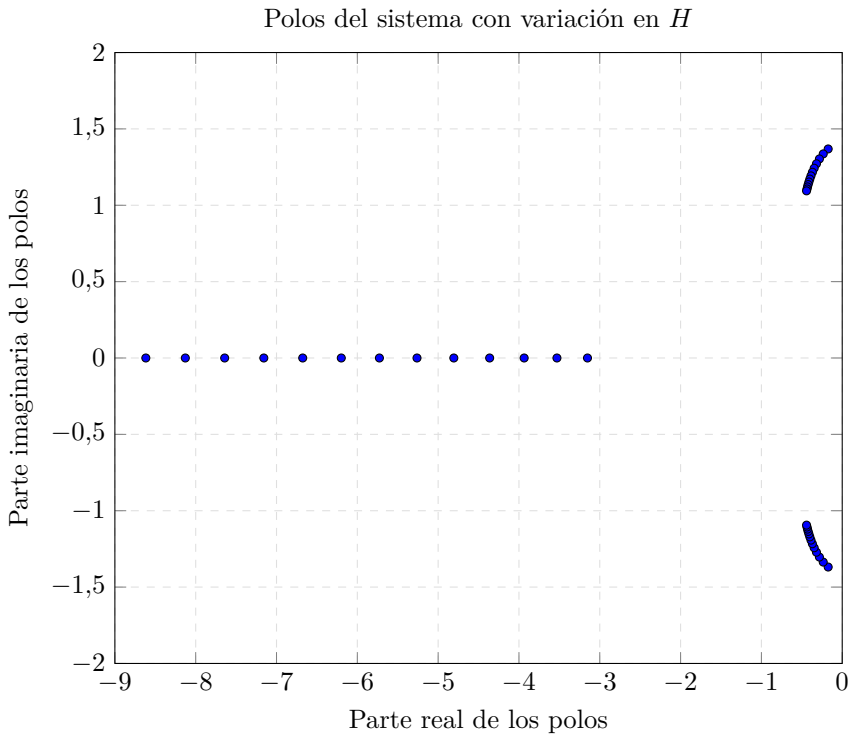


Figura 1. Lugar geométrico de las raíces del sistema realimentado al variar H entre 3 y 9.

Se observa de la figura que todos los polos se encuentra en el semiplano izquierdo, por lo que el sistema es estable para todo valor de H. El código 1 se muestra el código utilizado.

```
clear all

%{
Primer ejercicio
%}
Gs = tf([1, 1, 1], [1, 0.5, 0, 3]); %sistema de la gu a 04
Hs = tf([6], [1]);
feed = feedback(Gs, Hs);
polos = pole(feed);

%%
%{
Segundo ejercicio
%}

```

```
fig = figure;
hold on;
grid on;
theme(fig, 'light')
matriz_polos = [];
for c = 3:0.5:9
    i = 1;
    Hs = tf([c], [1]);
    sis = feedback(Gs, Hs);
    polos = pole(sis);
    x = real(polos) ; y = imag(polos);
    disp(x)
    plot(x, y, 'b.', MarkerSize=12)
    axis([-10, 1, -3, 3])
    title(['Polos del sistema con H =', num2str(c)])
    xlabel('Parte real de los polos')
    ylabel('Parte imaginaria de los polos')
    drawnow
    pause(0.1)
    i = i + 1;
end
```

Código 1: Código MATLAB para generar el lugar de raíces del sistema G

Aunque la compilación de ese código genera una animación, son demasiadas líneas para calcular cada polo, además de que generamos un bucle el cual es costoso computacionalmente. La opción más rápida es utilizar `rlocus(Gs)`.

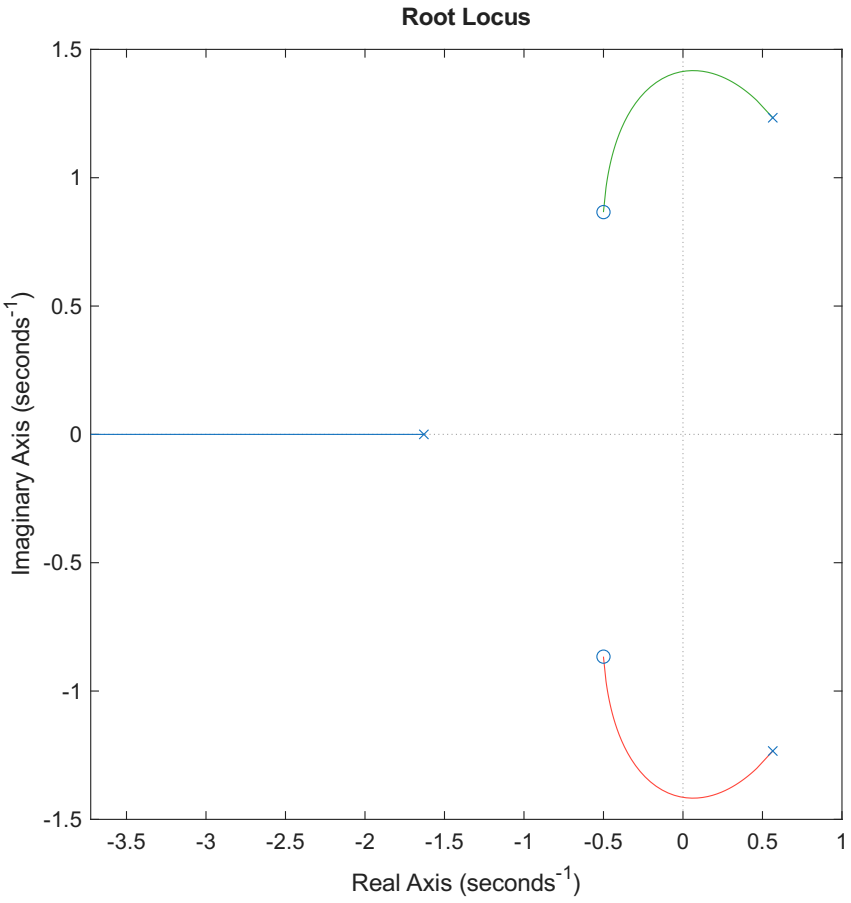


Figura 2. Lugar de raíces continuo, usando `rlocus(Gs)`

Para calcular la función de transferencia en lazo cerrado se utiliza la siguiente expresión:

$$\frac{Y(s)}{R(s)} = \frac{G(s)}{1 + G(s)H(s)} \tag{2}$$

Pero también es muy común utilizar la siguiente expresión:

$$\frac{Y(s)}{R(s)} = \frac{G(s)H(s)}{1 + G(s)H(s)} \tag{3}$$

Si adaptamos el código para este nuevo sistema, el lugar de raíces es:

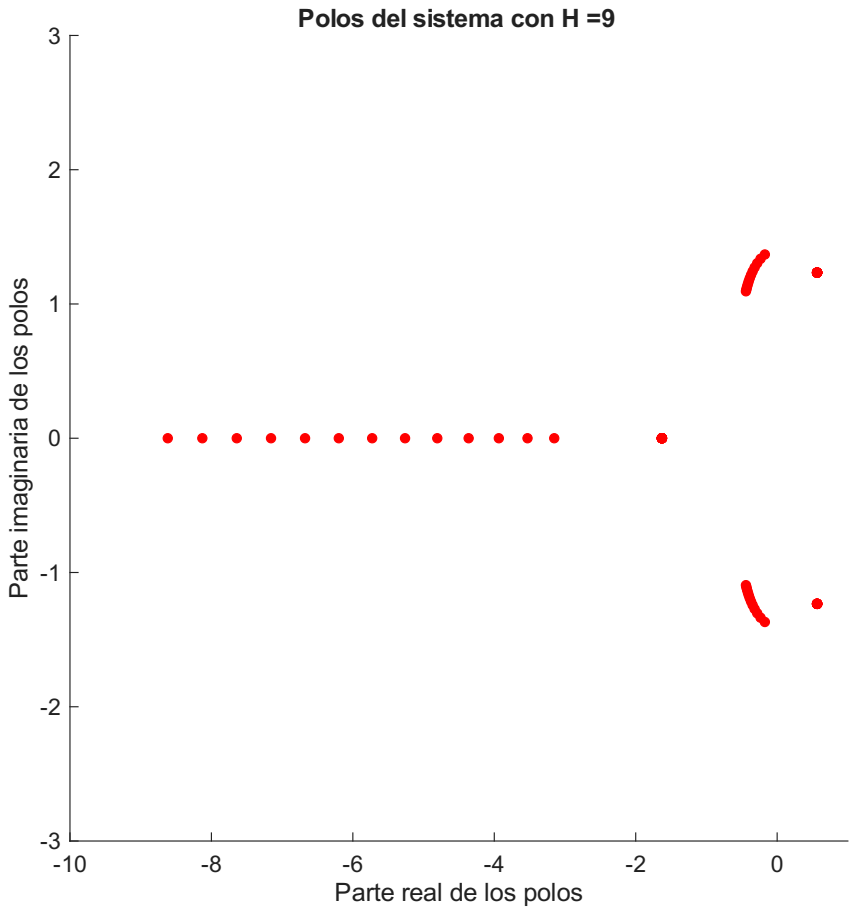


Figura 3. Lugar de raíces según la expresión (3).

No cambia nada con respecto a la figura anterior ya que el denominador no cambió. Los códigos usados en MATLAB se encuentran debajo:

```
clear all

%{
Primer ejercicio
%}
Gs = tf([1, 1, 1], [1, 0.5, 0, 3]); %sistema de la gu a 04
Hs = tf([6], [1]);
feed = feedback(Gs, Hs);
polos = pole(feed);

%%
%{
Segundo ejercicio
```

```
%}

% fig = figure;
% hold on;
% grid on;
% theme(fig, 'light')
% matriz_polos = [];
% for c = 3:0.5:9
%     i = 1;
%     Hs = tf([c], [1]);
%     sis = feedback(Gs, Hs);
%     polos = pole(sis);
%     x = real(polos) ; y = imag(polos);
%     % disp(x)
%     plot(x, y, 'b.', MarkerSize=12)
%     axis([-10, 1, -3, 3])
%     title(['Polos del sistema con H =', num2str(c)])
%     xlabel('Parte real de los polos')
%     ylabel('Parte imaginaria de los polos')
%     drawnow
%     pause(0.1)
%     i = i + 1;
% end

%{
El c digo ayuda a ubicar el lugar de ra ces.
Me genera una 'animaci n' a medida que el
el compensador aumenta de valor. Es estable
para todos los valores; todos se encuentran
en el semiplano izquierdo (estabilidad del
sistema).

Aunque el resultado es lindo, calcular
de esta forma las ra ces del sistema
es laborioso. Por lo tanto, la mejor opci n
es usar la funci n 'rlocus(Gs)' y me grafica
todo el lugar de ra ces de dicho sistema.
Aunque la funci n de rlocus es f cil , no
presenta la ventaja de cambiar el compensador
%}

% Nuevo Gs
fig2 = figure;
theme(fig2, 'light')
Gs1 = tf([1, 1], [1, 0.5, 0, 3]);
rlocus(Gs)




%%

%{
Implemento el nuevo sistema G*H/(1+G*H)
Es pr cticamente lo mismo que el sistema
anterior.
%}
fig3 = figure;
theme(fig3, 'light')
hold on;
for c = 3:0.5:9
```

```
    Hs = tf([c], [1]);
    new_sis = Gs*Hs/(1+Gs*Hs);
    polos = pole(new_sis);
    x = real(polos) ; y = imag(polos);
    plot(x, y, 'r.', MarkerSize=12)
    axis([-10, 1, -3, 3])
    title(['Polos del sistema con H =', num2str(c)])
    xlabel('Parte real de los polos')
    ylabel('Parte imaginaria de los polos')
    drawnow
    pause(0.1)
end
```

Código 2: Código MATLAB para graficar el lugar de raíces

Guía 12: Funciones en MATLAB

El siguiente código representa una función para calcular la raíz cuadrada de cualquier número:

```
clear all;
%{
Funciones
%}

function raiz_t = raiz(t)
    raiz_t = sqrt(t);
end
```

Ahora veamos el siguiente sistema:

$$G(s) = \frac{a}{bs^2 + cs + d} = \frac{k\omega^2}{s^2 + 2\zeta\omega s + \omega^2} \tag{4}$$

El cual puede ser escrito como:

$$G(s) = \frac{a/b}{s^2 + c/bs + d/b} \tag{5}$$

Se aprecia que $\omega^2 = d/b$, $k = a/b\omega^2$ y $\zeta = c/(2\omega)$. El siguiente código usa tres ejemplos:

```

function [k, w, z] = prototipo2(a, denom)
    b = denom(1);
    c = denom(2);
    d = denom(3);
    w = sqrt(d/b);
    k = a/b*w^2;
    z = c/(2*w);
end

%{
Para G1: no tiene sobrepaso ya que z = 1
%}
[k1, w1, z1] = prototipo2(16, [2, 4, 8]);

%{
Para G2: no tiene sobrepaso ya que z = 2
%}
[k2, w2, z2] = prototipo2(18, [1, 12, 9]);

%{
Para G3: no tiene sobrepaso ya que z = 1.6016
%}
[k3, w3, z3] = prototipo2(11, [1.7, 6.5, 7]);
```